Introduction

Distinct Subsequences shows up on LeetCode as problem 115, and it's tagged "hard" for a reason that has nothing to do with tricky syntax — it's hard because the brute force approach feels natural and is completely unworkable at scale. Interviewers reach for this problem when they want to see whether you can spot repeated subproblems inside a recursive tree and convert that insight into a table. That skill — recognizing overlapping work and building a grid to eliminate it — is the same skill behind edit distance, longest common subsequence, and a dozen other string-comparison problems that show up constantly in interviews.

It's also a problem with a real-world twin. Anywhere you need to count how many ways a shorter pattern can be extracted, in order, from a longer sequence, you're solving this exact structure — including in bioinformatics, where researchers count how many times a short genetic marker appears as a subsequence inside a much longer strand of DNA.

Problem Overview

You're given two strings, `s` and `t`. Your job is to count how many distinct ways you can delete characters from `s` so that what remains spells out `t` exactly, in the same order the characters appear in `t`. You're not allowed to reorder anything — only remove characters — and each unique set of positions you keep counts as one distinct way, even if the resulting letters look the same as another way you already counted.

The answer is guaranteed to fit inside a standard 32-bit signed integer, and both strings can be as long as 1,000 characters, which is the constraint that rules out any approach that tries to enumerate subsequences directly.

Example

Take `s = "rabbbit"` and `t = "rabbit"`.

At first glance, most people guess there's exactly one way to turn `rabbbit` into `rabbit`, since it looks like you're just removing "the extra b." But look closer at the three b's sitting in the middle of `rabbbit`. The target `rabbit` only needs two of them, so any pair of those three b's will do:

- Keep the 1st and 2nd b, drop the 3rd

- Keep the 1st and 3rd b, drop the 2nd
- Keep the 2nd and 3rd b, drop the 1st

That's three distinct deletions, and all three produce `rabbit`. The answer is 3.

A second example makes the boundary condition obvious: if `s = "abc"` and `t = "abcd"`, the answer is 0, because a subsequence can only remove characters from `s` — it can never add one. You cannot pull a longer string out of a shorter one, no matter how you choose the deletions.

Intuition

Don't start by thinking about grids or tables. Start by walking through `s` one character at a time and asking a single question at each position: does this character match the next character I still need from `t`?

- If it doesn't match, you have no choice — skip it and move on.
- If it does match, you have a real decision: use this character toward matching `t`, or skip it anyway and hope a later occurrence works out instead.

That second case is exactly why `rabbbit` produces three answers instead of one — at each of the three b's, you could use it or defer to a different one, and different combinations of those choices still land on a valid match.

Once you frame it as "count the ways to skip" plus "count the ways to use it when it matches," you have a recurrence. If you draw out the recursive call tree for this, you'll notice the same pair of positions in `s` and `t` gets revisited again and again from different branches. That repeated work is the actual bottleneck — not the logic itself, which is simple, but the fact that plain recursion recomputes the same answer thousands of times over.

Brute Force Solution

Idea: Recursively decide, character by character, whether to use or skip each letter of `s`, and count how many decision paths result in `t` being fully matched.

Algorithm: 1. Define a recursive function `count(i, j)` meaning "ways to form `t[j:]` from `s[i:]`." 2. If `j` reaches the end of `t`, that's one valid way — return 1. 3. If `i` reaches the end of `s` before `j` does, there's no way left — return 0. 4. If `s[i] == t[j]`, the count is `count(i+1, j+1) + count(i+1, j)` (use it, or skip it). 5. Otherwise, the count is just `count(i+1, j)` (forced skip).

Advantages: Directly mirrors the problem statement, easy to reason about correctness, no setup required.

Disadvantages: Recomputes identical subproblems `(i, j)` over and over. For `s = "rabbbit"` (7 characters), you're already looking at up to $2^7 = 128$ possible subsequences to check against `t`, and only 3 of them match. Scale that up to strings of length 1,000, and the recursion tree becomes exponential — completely infeasible.

Complexity: Time is $O(2^n)$ in the worst case, where n is the length of `s`. Space is O(n) for the recursion stack (or worse if you materialize subsequences).

Optimal Solution

The fix for repeated work is the fix for repeated work everywhere: cache it. Since `count(i, j)` only ever depends on `i` and `j`, build a 2D grid where the rows correspond to prefixes of `s` and the columns correspond to prefixes of `t`. Each cell `dp[i][j]` holds the number of ways to form the first `j` characters of `t` using the first `i` characters of `s`.

Base cases: - An empty target string has exactly one way to be formed from any prefix of `s`: delete everything. So the entire first column (`j = 0`) is filled with 1's. - If `s` is empty but `t` isn't, there's no way to form `t` at all. So the first row (`i = 0`), past the very first cell, is filled with 0's.

Filling the grid, row by row, left to right: - Every cell starts by inheriting the value directly above it — `dp[i-1][j]` — because skipping the current character of `s` is always an option. - If `s[i-1] == t[j-1]` (the characters at this row/column match), add in the diagonal cell `dp[i-1][j-1]` too, since using this character to match `t[j-1]` is now also on the table.

	""	r	a	b	b	i	t
""	1	0	0	0	0	0	0
r	1	1	0	0	0	0	0
a	1	1	1	0	0	0	0
b	1	1	1	1	0	0	0
b	1	1	1	2	1	0	0
b	1	1	1	3	3	0	0
i	1	1	1	3	3	3	0
t	1	1	1	3	3	3	3

By the bottom-right cell, the table reads 3 — matching the answer we found by hand.

Each row only ever reads the row directly above it, which means you don't need the full 2D table at all. You can collapse it down to a single 1D array of length `len(t) + 1`, updated from right to left so you

don't overwrite a value before you've read it. That drops the space complexity from O(m·n) to O(n), and it's usually the actual follow-up question interviewers ask — the recurrence itself is considered the easy half.

Python Code

```
def num_distinct(s: str, t: str) -> int:
    """Count distinct subsequences of s that equal t."""
    m, n = len(s), len(t)

    # dp[i][j] = ways to form t[:j] using s[:i]
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # An empty target has exactly one way to be formed: delete everything.
    for i in range(m + 1):
        dp[i][0] = 1

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            dp[i][j] = dp[i - 1][j]  # skip s[i-1]
            if s[i - 1] == t[j - 1]:
                dp[i][j] += dp[i - 1][j - 1]  # also use s[i-1]

    return dp[m][n]


def num_distinct_optimized(s: str, t: str) -> int:
    """Same recurrence, rolled down to O(n) space."""
    m, n = len(s), len(t)
    dp = [0] * (n + 1)
    dp[0] = 1

    for i in range(1, m + 1):
        # Right to left so dp[j-1] still holds the previous row's value.
        for j in range(n, 0, -1):
            if s[i - 1] == t[j - 1]:
                dp[j] += dp[j - 1]

    return dp[n]
```

Code Walkthrough

- `m, n = len(s), len(t)` — grab both lengths up front since we'll size the grid off them.
- `dp = [[0] * (n + 1) for _ in range(m + 1)]` — the grid is one row taller and one column wider than the strings themselves, to leave room for the empty-prefix cases.

- The first loop, `for i in range(m + 1): dp[i][0] = 1`, seeds every row's first column with 1, since matching an empty target always has exactly one way (delete everything).
- The nested loop walks the grid row by row, column by column. `dp[i][j] = dp[i - 1][j]` starts each cell as the "skip" option, inherited straight from the row above.
- `if s[i - 1] == t[j - 1]: dp[i][j] += dp[i - 1][j - 1]` adds in the "use this character" option only when the current letters actually match.
- `return dp[m][n]` — the bottom-right corner holds the answer for the full strings.
- The optimized version replaces the 2D grid with one array, iterating `j` backward so that `dp[j - 1]` still refers to the previous row's value at the moment it's read, rather than a value we've already updated in this pass.

Dry Run

Using `s = "rab"`, `t = "rb"` for a compact trace:

Initial row (`i = 0`): `[1, 0, 0]`

Row `i = 1`, `s[0] = 'r'`: - `j = 1`, `t[0] = 'r'` — matches. `dp[1][1] = dp[0][1] + dp[0][0] = 0 + 1 = 1` - `j = 2`, `t[1] = 'b'` — no match. `dp[1][2] = dp[0][2] = 0` - Row: `[1, 1, 0]`

Row `i = 2`, `s[1] = 'a'`: - `j = 1`, `t[0] = 'r'` — no match. `dp[2][1] = dp[1][1] = 1` - `j = 2`, `t[1] = 'b'` — no match. `dp[2][2] = dp[1][2] = 0` - Row: `[1, 1, 0]`

Row `i = 3`, `s[2] = 'b'`: - `j = 1`, `t[0] = 'r'` — no match. `dp[3][1] = dp[2][1] = 1` - `j = 2`, `t[1] = 'b'` — matches. `dp[3][2] = dp[2][2] + dp[2][1] = 0 + 1 = 1` - Row: `[1, 1, 1]`

Final answer: `dp[3][2] = 1` — there's exactly one way to pull "rb" out of "rab," which checks out since there's only one 'r' and one 'b' to choose from.

Complexity Analysis

Time: O(m·n), where m is the length of `s` and n is the length of `t`. Every cell in the grid is computed exactly once, in constant time, and there are (m+1)·(n+1) cells.

Space: O(m·n) for the full 2D table, or O(n) for the rolled-down single-row version, since each row only ever depends on the row directly above it.

These bounds are tight because you genuinely need to consider every pairing of a prefix of `s` with a prefix of `t` at least once — there's no way to skip a cell without risking missing a valid combination, since any cell could be the one that changes the final count.

Alternative Solutions

You could implement this with top-down memoization instead of a bottom-up table — write the same recursive function from the brute force section, but cache results in a dictionary keyed by `(i, j)`. It has identical time and space complexity to the 2D tabulation approach, and some engineers find it more intuitive to reason about since it mirrors the original recursive definition directly. The tradeoff is Python's function-call overhead makes it noticeably slower in practice than the iterative table, and it doesn't lend itself as naturally to the space optimization, since recursion doesn't process rows in a strict order you can roll down easily.

Edge Cases

- **Empty `t`:** Always returns 1, since deleting everything from `s` is itself one valid way to produce an empty string.
- **Empty `s`, non-empty `t`:** Always returns 0, since there's nothing to select from.
- **`t` longer than `s`:** Always returns 0. A subsequence can only remove characters, never add them, so a longer target can never be pulled out of a shorter source — this is easy to forget when you're focused on the matching logic instead of the length check.
- **Duplicate characters, like the three b's in `rabbbit`:** These are exactly what produce counts greater than 1, since multiple positions can independently satisfy the same required character.
- **`s` equals `t` exactly:** Returns 1, since there's exactly one way to select every character in order with nothing left to delete.

Common Mistakes

1. **Filling the first row with 1's instead of 0's (past the corner).** Only `dp[i][0]` should be 1 for all `i`; `dp[0][j]` for `j > 0` must be 0, since an empty `s` can't produce a non-empty `t`.
2. **Forgetting the "skip" contribution when characters match.** Even on a matching letter, you must still add `dp[i-1][j]`, not just `dp[i-1][j-1]` — using the match isn't mandatory.
3. **Iterating the rolled-down array left to right instead of right to left.** This causes you to read a value that's already been updated in the current pass, corrupting the count.

4. **Off-by-one errors indexing `s[i-1]` and `t[j-1]`** instead of `s[i]` and `t[j]`, since the grid is offset by one to make room for the empty-prefix row and column.
5. **Assuming the answer is always 1 when `t` is a literal substring of `s`.** Being contiguous doesn't matter — repeated characters anywhere in `s` can still multiply the count, as `rabbbit` demonstrates.

Interview Questions

- Can you reduce the space complexity from O(m·n) to O(n)? Walk through why iterating backward is required.
- What if you needed to return one example of a valid subsequence selection, not just the count?
- How would this change if `s` and `t` could contain wildcard characters that match anything?
- Can you adapt this same table structure to solve Longest Common Subsequence or Edit Distance?
- What's the time complexity if you tried to solve this by generating all subsequences directly, and why does that not scale?

Similar Problems

- **LeetCode 72, Edit Distance** — same 2D grid pattern, but counting the minimum operations (insert, delete, replace) instead of subsequence matches.
- **LeetCode 1143, Longest Common Subsequence** — nearly identical grid setup, but tracking the length of the longest shared subsequence rather than the count of ways to form one exactly.
- **LeetCode 583, Delete Operation for Two Strings** — reframes the edit distance idea around deletions only, which connects directly back to how subsequences are formed here.
- **LeetCode 97, Interleaving String** — a different flavor of the same "build a grid over two string indices" technique.

Key Takeaways

Distinct Subsequences is really a lesson in recognizing repeated work inside a recursive tree and eliminating it with a table. The recurrence itself — skip the source character, or use it when it matches and add both options together — is straightforward once you trace it by hand on an example like `rabbbit`. The real skill being tested is converting that recursive insight into an O(m·n) table, and then recognizing that the table can be compressed to a single row, since each row only ever depends on the one directly above it. Any time you're comparing two ordered sequences and counting matches

between them — words, DNA, log entries — this same grid shape, one axis per sequence, is the tool to reach for.

Watch the Video

For the full walkthrough — including the recursive call tree, the timed brute-force-vs-optimal comparison, and two quick quizzes to check your understanding of the recurrence — watch the video here: {LINK}

About the Series

This breakdown is part of Fun with Learning Technology, a series that works through LeetCode problems, core computer science concepts, and everyday developer tools the way you'd want a colleague to explain them — tracing the intuition first, building up to the code, and never skipping the "why."

Call To Action

If this walkthrough made Distinct Subsequences click, the full video has the visual grid fill-in and the timed benchmark that makes the O(m·n) improvement concrete — give it a watch and a like on YouTube. Subscribe there for the next problem in the series, and if you want these breakdowns delivered as writeups like this one, subscribe to the Substack. Drop a comment with your own approach or a problem you'd like covered next, and share this with anyone prepping for interviews.

Quick Review

Overlapping substring counting → which DP pattern?

Two-sequence subsequence-counting DP: dp[i][j] = ways s[:i] forms t[:j], think 'match or skip' recurrence.

Time complexity for counting distinct subsequences (s len n, t len m)?

O(n*m) — fill one DP table cell per (s,t) prefix pair.

Space complexity, and how to shrink it?

O(n*m) naive; O(m) with rolling 1D array since each row only needs the previous row.

Common bug when coding the recurrence?

Forgetting dp[i][0]=1 for all i (empty target always matches once) — breaks base case, undercounts everything.

Why can't you just check if t is a subsequence of s (two pointers)?

That only answers yes/no; counting ALL ways requires branching at every match, which two-pointer greedy discards.

Follow-up: how does matching s[i]==t[j] update dp?

dp[i][j] = dp[i-1][j] (skip s[i]) + dp[i-1][j-1] if chars equal (also use s[i] as match).

Distinct Subsequences — free Python cheat sheet